A4 · 学生实操手册 —— SOLO 迭代:进阶控件 + 管理员端 + 组合搜索 + 服务端监控窗口

> **对应**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元④ **用法**:跟着一步一步做,每一步都有【操作】【预期产出】【✅ 怎么算做好】【卡住怎么办】。老师带做时你同步做,课后自己做也照这份来。 **前置**:单元②③已跑通——服务端(监听 9000、每客户端一线程)+ 读者端(登录 / 查书 / 借书事务 / 还书)。本单元在这个基础上继续加功能,**前面的代码一个字都不用推倒重来**。

今天要做什么(先读这一段)

前三个单元,这套 C/S 图书馆的**主干链路**已经通了:`客户端 → Socket → 服务端 → JDBC → 库`,而且读者能用了。今天是在**现有基础上加功能**——这叫**迭代式开发**,做法始终是三步:**说清需求(复制 prompt)→ AI 加这一块 → 你跑通 + 逐条验收。**

但**动手加功能前,先花前半程从零补三块你还没学过的进阶知识**——不补,今天生成的界面和监控窗口你会看不懂、审不动:

要补的	用在哪	本手册对应
Swing 进阶控件(JTable / JDialog / JTabbedPane / 工具栏)	管理员端的表格、弹窗、选项卡	热身 A
EDT(事件分发线程)	监控窗口:别的线程改界面要切回 EDT	热身 B
发布 - 订阅	监控窗口:把各线程事件汇总到一个窗口	热身 C

补完三块基础,再进图书馆三步实操:

步骤	你要做的事	产出
热身 A/B/C	跑通三个最小 demo(进阶控件 / EDT / 发布订阅)	亲手见过它们长什么样,后面审 AI 代码不虚
实操 1	SOLO 生成管理员端 + 审查	管理员登录后能管图书 / 看读者 / 看超期,过了三条审查
实操 2	组合搜索(+ 分页可选)	图书查询能按关键词 + 分类组合筛
实操 3	⭐ SOLO 生成服务端监控窗口 + 审查	监控窗口能看端口/在线数/日志、能启停,过了两条铁律审查

> 💡 **本单元两个审查重点**(你的判断力发挥价值的地方):
>
> - 管理端:表格只读、删除有确认框、请求走异步。
> - **监控窗口:跨线程更新界面必须切回 EDT(`SwingUtilities.invokeLater`);日志绝不能记密码。** AI 图快很容易漏这两条,你要能看出来、能让它补。

【热身:先补三块进阶基础】

> 这三个 demo 都很小、能直接跑。**亲手跑一遍**,今天要审的界面和监控窗口你就全懂了。看不懂的地方当场让 AI 用中文逐行解释。

热身 A · Swing 进阶控件(JTable / JDialog / JTabbedPane)

这一步在做什么

管理员端要做三件读者端没做过的事,正好对应三个新控件:**把一批书显示成表格(JTable)、点新增弹表单(JDialog)、把几块功能分页(JTabbedPane)**。先各跑一个最小 demo。

A.1 · JTable + DefaultTableModel(把数据显示成只读表格)

【操作】让 SOLO 生成并运行下面这个 demo(或直接对照母本 Part 1.2 手敲):

```
用纯 Java 11 Swing 写一个最小 demo:一个 JTable 显示三本书(ID/书名/作者),
表格只读(重写 DefaultTableModel 的 isCellEditable 返回 false),
选中某一行时把该行书名打印到控制台。表格包一层 JScrollPane。
```

【预期产出(对照母本 Part 1.2)】

```
DefaultTableModel model = new DefaultTableModel(
        new Object[]{"ID", "书名", "作者"}, 0) {
    @Override public boolean isCellEditable(int r, int c) {
        return false;   // 表格只读:不让用户在格子里直接改
    }
};
model.addRow(new Object[]{1, "Java 编程思想", "Bruce Eckel"});   // 加行
model.addRow(new Object[]{2, "算法导论", "Cormen"});

JTable table = new JTable(model);
table.getSelectionModel().addListSelectionListener(e -> {
    int row = table.getSelectedRow();                // 取选中行号(没选返回 -1)
    if (row >= 0) System.out.println("你选中了:" + model.getValueAt(row, 1));
});
frame.add(new JScrollPane(table));                   // 表格包一层 JScrollPane 才能滚动
```

【✅ 怎么算做好】跑起来看到一张三行的表格;点某一行,控制台打印"你选中了:XXX";在格子里双击**改不动**(只读生效)。

【记住这一句】**数据在 model 里,表格只是把 model 画出来。** 加行改行都操作 model;取选中行两步走——先 `getSelectedRow()` 拿行号,再 `getValueAt(行,列)` 拿值。**表格只读那句 `isCellEditable` 返回 false 是一会儿审查管理端的第一条看点。**

A.2 · JDialog(弹一个新增/编辑表单)

【操作】让 SOLO 生成并运行:

```
用纯 Java 11 Swing 写一个 JDialog demo:主窗上两个按钮"新增"和"编辑《算法导论》"。
点"新增"弹一个模态(modal)对话框、书名输入框是空的;
点"编辑"弹同样的对话框但书名预填"算法导论"。
对话框里一个"保存"按钮,点了打印"新增:xxx"或"更新:xxx"再关闭。
关键:同一个方法 openEdit(bookTitle),传 null=新增、传书名=编辑。
```

【预期产出(对照母本 Part 1.3)】

```
// book 传 null = 新增;传一个书名 = 编辑(预填)
JDialog dlg = new JDialog(owner,
        bookTitle == null ? "新增图书" : "编辑图书", true);  // true = 模态,主窗点不了
JTextField titleField = new JTextField(12);
if (bookTitle != null) titleField.setText(bookTitle);        // 编辑时预填

JButton save = new JButton("保存");
save.addActionListener(e -> {
    System.out.println((bookTitle == null ? "新增" : "更新") + ":" +
titleField.getText());
    dlg.dispose();   // 关掉弹窗
});
dlg.setVisible(true);   // 模态窗会在这里"卡住",等用户关掉才继续
```

【✅ 怎么算做好】点"新增"弹空表单;点"编辑"弹一个已预填书名的表单。对话框开着时主窗点不动(模态)。点保存,控制台打印对应文字并关窗。

【记住这一句】弹窗表单就一个套路——**传 null 是新增、传对象是编辑(预填)**,保存时按这个判断调不同接口。今天 AI 生成的 `BookEditDialog` 就是这么写的。

A.3 · JTabbedPane + 工具栏 + mini-CRUD(拼起来)

【操作】让 SOLO 生成并运行下面这个 mini-CRUD(它把表格 + 工具栏 + 弹窗 + 删除确认拼到一起):

```
用纯 Java 11 Swing 写一个最小 CRUD demo:上面一排工具栏(新增/删除两个按钮),
下面一个只读 JTable(ID/书名)。点"新增"用 JOptionPane.showInputDialog 输书名、加一行;
点"删除"先判断有没有选中行(没有就提示),再用 showConfirmDialog 确认,点是才删。
```

【预期产出(对照母本 Part 1.6)】

```
static DefaultTableModel model = new DefaultTableModel(new Object[]{"ID","书名"},
0) {
    @Override public boolean isCellEditable(int r, int c) { return false; }  //
只读
};
JButton add = new JButton("新增");
add.addActionListener(e -> {                                  // 点新增 → 弹表单
    String name = JOptionPane.showInputDialog("输入书名:");
    if (name != null && !name.isEmpty()) model.addRow(new Object[]{nextId++,
name});
});
JButton del = new JButton("删除");
del.addActionListener(e -> {                                  // 点删除 → 先确认
    int row = table.getSelectedRow();
    if (row < 0) { JOptionPane.showMessageDialog(null, "请先选中一行"); return; }
    int r = JOptionPane.showConfirmDialog(null, "确定删除?", "确认",
JOptionPane.YES_NO_OPTION);
    if (r == JOptionPane.YES_OPTION) model.removeRow(row);
});
JPanel bar = new JPanel(new FlowLayout(FlowLayout.LEFT));
bar.add(add); bar.add(del);
JPanel panel = new JPanel(new BorderLayout());
panel.add(bar, BorderLayout.NORTH);
panel.add(new JScrollPane(table), BorderLayout.CENTER);
```

`JTabbedPane`(选项卡)也顺手记一下,今天管理端主窗要用,关键就两句:

```
JTabbedPane tabs = new JTabbedPane();                  // 建选项卡容器
tabs.addTab("图书管理", new JLabel("这里放图书表格 + 工具栏"));    // 加一页
frame.setContentPane(tabs);
```

【✅ 怎么算做好】表格 + "新增/删除"两个按钮;点新增弹输入框加一行,点删除先弹确认框再删。

【记住这一句】**JTable 显示 + 工具栏按钮 + 弹窗表单 + 删除确认——管理端一个面板就这么点东西。**今天 AI 生成的 `BookManagePanel` 比这个多的,只是"数据从网络来""表单更漂亮",**骨架完全一样**。你现在有了这个 demo 的底子,待会儿审 AI 写的界面门儿清。

热身 B · EDT(事件分发线程):谁能碰界面

这一步在做什么

监控窗口的日志和在线数,是服务端**各个客户端处理线程**产生的,它们要显示到界面上。Swing 有条铁律:**界面只能由 EDT(事件分发线程)这一条线程碰**;别的线程要改界面,必须用 `SwingUtilities.invokeLater` 切回 EDT。先跑一个 demo 见识这条铁律。

操作

让 SOLO 生成并运行(母本 Part 3.4):

```
用纯 Java 11 Swing 写一个 demo:一个窗口里有一个 JLabel 显示"计数:0"。
开一个后台线程,每秒把数字 +1 刷到 JLabel 上。
关键:后台线程不能直接 label.setText(...),必须用 SwingUtilities.invokeLater 切回 EDT。
```

预期产出(对照母本 Part 3.4)

```
new Thread(() -> {                          // 后台线程,不是 EDT
    for (int i = 1; i <= 100; i++) {
        final int n = i;
        // ⭐ 后台线程不能直接 label.setText(...),必须切回 EDT
        SwingUtilities.invokeLater(() -> label.setText("计数:" + n));
        try { Thread.sleep(1000); } catch (InterruptedException ignore) {}
    }
}).start();
```

✅ 怎么算做好

窗口上的数字每秒 +1。重点不是"数字会动",而是理解:那个 `new Thread(...)` 是**后台线程,不是 EDT**,它改 `label` 必须用 `invokeLater` 切回 EDT。

记住这一句

别的线程不许碰界面,要碰先 `invokeLater` 回到 EDT。 这和单元③"耗时请求放后台不卡界面"是**同一道理的两面**:那边"别在 EDT 干耗时活",这边"别在非 EDT 改界面"。**监控窗口的 `onLog` / `onOnlineChange`,正是这种场景——这就是一会儿审查的头号看点。**

(可选)让 AI 当老师

问它:"如果这里不用 `invokeLater`、直接 `label.setText(...)` 会怎样?为什么复杂界面会偶发错乱/卡死?" 把它的解释记下来。

热身 C · 发布 - 订阅:到处产生的事件汇到一个窗口

这一步在做什么

监控窗口的难点:日志和在线数**到处产生**(各个客户端线程),却要显示到**一个**窗口。让每个线程都直接持有窗口去改它,耦合太死。解法是加个"中间人"——**发布 - 订阅(广播站 / 订阅号)**:线程只管"喊一声",窗口只管"听到就显示",中间的 `ServerContext` 把两边接上。先跑一个极简"广播站"。

操作

让 SOLO 生成并运行(母本 Part 4.3):

```
用纯 Java 11 写一个最小的发布-订阅 demo(不用界面):
- 一个订阅接口 Listener,方法 onEvent(String);
- 一个 Broadcaster(发布者):有 setListener 登记订阅者、有 publish(msg) 先打印"[广播站记录]"再通知订阅者(为 null 就跳过);
- main 里注册一个订阅者(打印"[订阅者收到]"),然后 publish 两条事件。
```

预期产出(对照母本 Part 4.3)

```
interface Listener { void onEvent(String msg); }          // ① 订阅接口

static class Broadcaster {                                 // ② 发布者 / 中枢
    private Listener listener;
    void setListener(Listener l) { listener = l; }         // 订阅者来登记
    void publish(String msg) {
        System.out.println("[广播站记录] " + msg);         // 先自己记一笔(相当于打日志)
        if (listener != null) listener.onEvent(msg);       // 再通知订阅者(没有就跳过)
    }
}
// ③ 订阅者:实现 Listener,注册进去
bus.setListener(msg -> System.out.println("[订阅者收到] " + msg));
bus.publish("有人登录了");
bus.publish("有人借了书");
```

✅ 怎么算做好

每 `publish` 一条,先打印"[广播站记录]..."(发布者自己记),再打印"[订阅者收到]..."(通知到订阅者)。**试一下:注释掉 `setListener` 那行再跑——** `publish` 照样打印"[广播站记录]",只是没有"[订阅者收到]"。这正好对应"**监控窗口没开时,日志照样打到控制台**"。

对号入座(今天的真名)

最小模型	今天的真名	职责
发布者 / 中枢	`ServerContext`	`log()` / `clientConnected()` / `clientDisconnected()`
订阅接口	`ServerContext.Listener`	`onLog(String)` / `onOnlineChange(int)`
订阅者	`ServerMonitorFrame`	实现 Listener,构造时 `setListener(this)` 注册

> 💡 **在线计数为什么用 `AtomicInteger`?** 在线数是"多个客户端线程同时 ±1"的,普通 `int online++` 在多线程下会数错(两个线程同时读到 5、各 +1、都写回 6,丢了一次)。用 `AtomicInteger` 的 `incrementAndGet()` / `decrementAndGet()`,保证并发下每次 ±1 都不出错。记住"**并发 ±1 用原子类**"。

> ✅ **三块基础补完,现在进图书馆实操。** 你已经亲手跑过表格 / 弹窗 / 选项卡 / EDT / 发布订阅——待会儿 AI 生成的东西,你都见过它们的最小版本。

【实操】

实操 1 · SOLO 生成管理员端

这一步在做什么

管理员登录后进的是**另一个主窗**(`AdminMainFrame`),里面用选项卡(热身 A 的 JTabbedPane)装三块:图书管理、读者管理、借阅记录 / 超期统计。图书管理是重点——一个 JTable 列表 + 工具栏(新增 / 编辑 / 删除 / 刷新 / 搜索),新增和编辑弹一个 JDialog 表单,删除要有确认框。这些都和读者端一个套路,只是业务从"查 / 借"换成了"增删改"。

步骤 1.1 · 复制这段 prompt 给 Trae SOLO

【操作】在 SOLO 对话框**原样复制**输入并回车(取自抓手2 单元④ Step1):

```
基于现有代码,做管理员端主窗(选项卡):
- 图书管理:JTable + 工具栏(新增/编辑/删除/刷新/搜索),新增/编辑用 JDialog 表单,协议
BOOK_ADD|json、BOOK_UPDATE|json、BOOK_DELETE|id(服务端走 BookDao),删除有确认框;
- 分类管理、读者管理(READER_LIST)、借阅记录/超期统计(BORROW_RECORD)各一个选项卡。
只加管理员端,别动读者端。
```

【✅ 怎么算做好】Trae 生成了客户端 `AdminMainFrame`、`BookManagePanel`、`BookEditDialog`、`ReaderManagePanel`、`BorrowRecordPanel`,服务端 `ClientHandler` 里多了 `BOOK_ADD` / `BOOK_UPDATE` / `BOOK_DELETE` / `READER_LIST` / `BORROW_RECORD` 几条分支,而且**没动读者端**。

【卡住怎么办】

- **AI 把读者端也改了**:回一句"你改动了读者端的 XX,请恢复,这一步只加管理员端"。
- **一次生成太多、乱了**:让它分两步——"先只做管理员主窗 + 图书管理选项卡,读者管理和借阅记录下一步。"

步骤 1.2 · 生成后代码"该长什么样"(对照答案版)

打开 AI 生成的文件,对照下面的答案版片段看(不用一字不差,方向对上就行):

① **管理员主窗:三选项卡**(答案版 `client/ui/AdminMainFrame.java`)——就是热身 A 的 `JTabbedPane`:

```
JTabbedPane tabs = new JTabbedPane();
tabs.addTab("图书管理", new BookManagePanel());
tabs.addTab("读者管理", new ReaderManagePanel());
tabs.addTab("借阅记录 / 超期统计", new BorrowRecordPanel());
setContentPane(tabs);
```

② **图书管理:表格只读 + 工具栏**(答案版 `client/ui/BookManagePanel.java`)——就是热身 A 的 JTable + 工具栏:

```
// 表格只读:重写 isCellEditable 返回 false,数据不在表格里直接乱改
new DefaultTableModel(
        new Object[]{"ID", "ISBN", "书名", "作者", "出版社", "分类", "馆藏", "库存"},
0) {
    @Override public boolean isCellEditable(int r, int c) { return false; }
};
add.addActionListener(e -> openEdit(null));   // null = 新增
edit.addActionListener(e -> editSelected());
del.addActionListener(e -> deleteSelected());
search.addActionListener(e -> load());
```

③ **删除:先确认框,再异步删**(答案版 `client/ui/BookManagePanel.java`):

```
int r = JOptionPane.showConfirmDialog(this,
        "确定删除《" + b.getTitle() + "》吗?", "确认删除",
JOptionPane.YES_NO_OPTION);
if (r != JOptionPane.YES_OPTION) return;
Async.run(this, () -> { Api.deleteBook(b.getId()); return null; }, v -> {
    JOptionPane.showMessageDialog(this, "删除成功", "成功",
JOptionPane.INFORMATION_MESSAGE);
    load();
});
```

④ **新增 / 编辑对话框**(答案版 `client/ui/BookEditDialog.java`)——就是热身 A 的 JDialog:`book == null` 是新增,否则编辑并预填,保存走 `Api.addBook` / `Api.updateBook`:

```
super(owner, book == null ? "新增图书" : "编辑图书", true);    // modal 对话框
if (book != null) preload(book);                                 // 编辑时表单预填
```

保存动作(编辑与否由 `editing == null` 判断):

```
Async.run(this, () -> {
    if (editing == null) Api.addBook(b);
    else Api.updateBook(b);
    return null;
}, v -> {
    saved = true;
    JOptionPane.showMessageDialog(this, "保存成功", "成功",
JOptionPane.INFORMATION_MESSAGE);
    dispose();
});
```

⑤ **借阅记录 / 超期统计**(答案版 `client/ui/BorrowRecordPanel.java` + `server/dao/BorrowDao.java`):

```
// 客户端:一个"只看超期未还"复选框,协议 BORROW_RECORD|0/1
private final JCheckBox onlyOverdue = new JCheckBox("只看超期未还");
Async.run(this, () -> Api.borrowRecord(onlyOverdue.isSelected()), this::fill);

// 服务端:只看超期的 SQL 条件(未还 + 应还时间已过)
if (onlyOverdue) {
    sql.append("WHERE br.status = 'borrowed' AND br.due_time < NOW() ");
}
```

⑥ **服务端路由**(答案版 `server/ClientHandler.java` 的 switch 里,今天新增这几条分支):

```
case Protocol.BOOK_ADD:      return doBookAdd(p);
case Protocol.BOOK_UPDATE:   return doBookUpdate(p);
case Protocol.BOOK_DELETE:   return doBookDelete(p);
case Protocol.READER_LIST:   return doReaderList();
case Protocol.BORROW_RECORD: return doBorrowRecord(p);
```

【✅ 怎么算做好(实操 1 的功能验收点)】起服务端 → 起客户端 → 用 `admin/123456` 登录 → 进的是**管理端主窗**;图书管理能**新增 / 编辑 / 删除 / 搜索 / 刷新**(新增编辑弹表单、删除有确认框);借阅记录勾"只看超期未还"能筛出超期记录;读者管理能列出全部读者。

【卡住怎么办】

- **登录后进的还是读者端**:登录响应 `OK|role|realName|userId` 的 `role` 要分流——`admin` 打开 `AdminMainFrame`,`reader` 打开 `ReaderMainFrame`(单元③已做,若被覆盖让 AI 恢复)。
- **删除没确认框直接删了**:纠错三步法——"图书删除请先弹 `JOptionPane.showConfirmDialog` 确认,点是才删,只改这一处。"
- **新增/编辑后列表不刷新**:对话框 `isSaved()` 为真时,面板要重新 `load()` 一次。

步骤 1.3 · ⭐ 审查:管理端三条(自己找)

【操作】打开 AI 生成的 `BookManagePanel`,自己找这三点在哪:

审查点	怎么查	对的样子
🟡 表格只读	`DefaultTableModel` 有没有重写 `isCellEditable` 返回 false(热身 A 见过)	有
🟡 删除确认	删除前有没有 `showConfirmDialog`	有
🟡 异步请求	增删改查有没有走 `Async.run`(不在界面线程/EDT 上发网络请求,热身 B 的道理)	有

【✅ 怎么算做好】三点你都找到了、都在。找不到的那条,用纠错三步法让 AI 补。

> 💬 做完这一步你会发现:管理端这几个面板,和读者端**一个套路**(JTable + Async + 服务端 Handler→Service→Dao),新东西只有一个 JDialog 表单。**这就是"主干打通后,加功能是复制,不是重造"**;而且早上补的表格/弹窗/选项卡,现在全用上了。

实操 2 · 组合搜索(+ 分页可选扩展)

这一步在做什么

图书查询要能**同时按关键词(书名 / 作者模糊)+ 分类**筛。核心在服务端:**按条件动态拼 SQL**,并且关键词用**占位符** `LIKE ?`(不是字符串拼接)防注入。这是单元①写进 PRD 的编码规范,今天在搜索里落地。

步骤 2.1 · 看懂组合搜索(答案版已具备)

【操作】打开答案版 `server/dao/BookDao.java` 的 `query` 方法,重点看这段:

```
StringBuilder sql = new StringBuilder(
        "... FROM t_book b LEFT JOIN t_category c ON b.category_id = c.id WHERE 1
= 1 ");
if (keyword != null && !keyword.trim().isEmpty()) {
    sql.append("AND (b.title LIKE ? OR b.author LIKE ?) ");    // 占位符,防注入
    String like = "%" + keyword.trim() + "%";
    params.add(like);
    params.add(like);
}
if (categoryId > 0) {
    sql.append("AND b.category_id = ? ");
    params.add(categoryId);
}
```

两个要点:

- `WHERE 1 = 1` **+ 按条件 append**:关键词 / 分类有就加、没就不加,拼出来的 SQL 永远合法。
- `LIKE ?` **用占位符**:`%关键字%` 作为**参数**传进去,`PreparedStatement` 防注入——绝不把用户输入拼进 SQL 字符串。

【✅ 怎么算做好】图书查询同时输关键词、选分类,能筛出既匹配关键词、又属于该分类的书;两个都空 = 查全部。

步骤 2.2 · (可选)加分页

【操作】想加分页,给 SOLO 这段(抓手2 单元④ Step2):

```
给图书查询加价格/分类/关键词组合搜索和分页:服务端 BOOK_QUERY 支持 page/size 返回列表+总数;
客户端列表加上一页/下一页。只改图书查询相关。
```

理解分页的两件事:

- 服务端 SQL 末尾加 `LIMIT ? OFFSET ?`(`offset = (page-1)*size`),另用一条 `SELECT COUNT(*)` 查总数,响应带回 `{ 列表, 总数 }`。
- 客户端记住当前 `page`,"上一页 / 下一页"改 `page` 再重查;总页数 = `ceil(总数 / size)`。

【卡住怎么办】

- **翻页越界**:`page` 最小 1、最大不超过总页数;到边界把对应按钮 `setEnabled(false)`。
- **改分页把借还书搞乱了**:强调 prompt 里的"只改图书查询相关",AI 动了别处就让它恢复。

> 💡 分页的核心认知:**数据多的时候,不要一次全查回来**,后端只查这一页。我们种子数据少看不出效果,但真实系统几万条记录,分页是必备的。

实操 3 · ⭐ SOLO 生成服务端监控窗口

这一步在做什么

我们的服务端是个**后台应用服务器**,本身**没有界面**——打成 jar 双击,好像"什么也没发生"。今天给它加一个监控窗口:**监听端口、运行状态、在线客户端数、实时日志、启停按钮**(顶部几个 JLabel、中间一个只读 JTextArea、底部两个按钮,都是热身 A 学过的控件)。

做法是一个极简的**发布 - 订阅**(热身 C):服务端各处产生的事件(客户端接入 / 断开、每条请求的命令名 → OK/ERR)统一交给一个 `ServerContext`(集中日志 + 在线计数),监控窗口订阅它、把事件显示出来。**因为这些事件是各个客户端处理线程触发的,所以窗口更新界面时必须切回 EDT(热身 B)。**

步骤 3.1 · 复制这段 prompt 给 Trae SOLO

【操作】在 SOLO 对话框**原样复制**输入并回车(取自抓手2 单元④ Step3):

```
给服务端加一个监控窗口 ServerMonitorFrame(Swing),作为服务端 jar 的启动入口:
- 显示监听端口、运行状态、在线客户端数、实时日志(每次客户端接入/登录/借书打一行)、启动/停止按钮;
- 加一个 ServerContext(集中日志 + 在线计数,发布订阅),ClientHandler 接入/断开时在线数±1、每条请求记"命令→OK/ERR"(不记密码);
- LibraryServer 改成可 start()/stop()。保留一个控制台 main 便于无界面运行。
```

【✅ 怎么算做好】Trae 新增了 `server/ServerContext.java`、`server/ui/ServerMonitorFrame.java`,改了 `server/LibraryServer.java`(可 start/stop)和 `server/ClientHandler.java`(接入 / 断开上报、每请求记命令名)。

步骤 3.2 · 生成后代码"该长什么样"(对照答案版)

① **发布中枢** **`ServerContext`**(答案版 `server/ServerContext.java`)——就是热身 C 那个"广播站"的真实版:

```
public interface Listener {
    void onLog(String line);           // 有一行新日志
    void onOnlineChange(int online);   // 在线客户端数量变化
}
private static volatile Listener listener;
private static final AtomicInteger ONLINE = new AtomicInteger(0);    // 并发计数用原子类

public static void log(String msg) {
    String line = LocalTime.now().format(TF) + "  " + msg;
    System.out.println(line);                  // 控制台模式也能看到
```

```
    Listener l = listener;
    if (l != null) l.onLog(line);          // 通知订阅者(监控窗口)
}
public static void clientConnected() {
    int n = ONLINE.incrementAndGet();      // 在线 +1
    Listener l = listener;
    if (l != null) l.onOnlineChange(n);
}
public static void clientDisconnected() {
    int n = ONLINE.decrementAndGet();      // 在线 -1
    Listener l = listener;
    if (l != null) l.onOnlineChange(n);
}
```

② 监控窗口 `ServerMonitorFrame` 实现 Listener,回调里切 EDT(答案版 `server/ui/ServerMonitorFrame.java`)——就是热身 B 的 EDT 铁律落地:

```
public class ServerMonitorFrame extends JFrame implements ServerContext.Listener
{
    // 构造时:订阅服务端事件 + 自动启动服务
    ServerContext.setListener(this);
    onOnlineChange(ServerContext.online());
    LibraryServer.start();

    @Override
    public void onLog(String line) {
        SwingUtilities.invokeLater(() -> {                  // ← 头号看点:切回 EDT
            logArea.append(line + "\n");
            logArea.setCaretPosition(logArea.getDocument().getLength()); // 自动滚
到底
        });
    }

    @Override
    public void onOnlineChange(int online) {
        SwingUtilities.invokeLater(() -> onlineLabel.setText("在线客户端: " +
online));
    }
}
```

启停按钮:

```
startBtn.addActionListener(e -> { LibraryServer.start(); refreshStatus(); });
stopBtn.addActionListener(e -> { LibraryServer.stop(); refreshStatus(); });
```

③ 服务端可 start()/stop()(答案版 `server/LibraryServer.java`):

```
public static synchronized void start() {
    if (running) return;
    serverSocket = new ServerSocket(PORT);
```

```
    running = true;
    ServerContext.log("[Server] 启动成功,监听端口 " + PORT + ",等待客户端连接...");
    acceptThread = new Thread(() -> {
        while (running) {
            try {
                Socket client = serverSocket.accept();           // 阻塞等客户端
                new Thread(new ClientHandler(client)).start();   // 每客户端一线程
            } catch (IOException e) {
                if (running) ServerContext.log("[Server] accept 异常: " +
e.getMessage());
                break;
            }
        }
    }, "accept-loop");
    acceptThread.start();
}
public static synchronized void stop() {
    if (!running) return;
    running = false;
    try { if (serverSocket != null) serverSocket.close(); } catch (IOException
ignore) {}
    // close() 让阻塞中的 accept 抛异常,从而退出循环
}
```

④ **`ClientHandler` 上报接入 / 断开 + 脱敏日志**(答案版 `server/ClientHandler.java`):

```
ServerContext.clientConnected();                // 在线数 +1
ServerContext.log("客户端接入 " + addr);
...
// 只记录命令名(不记参数,避免把密码等敏感信息打进日志)+ 结果
String cmd = line.contains("|") ? line.substring(0, line.indexOf('|')) : line;
ServerContext.log(addr + "  " + cmd + " → " + (response.startsWith("OK") ? "OK" :
"ERR"));
...
finally {
    try { socket.close(); } catch (IOException ignore) {}
    ServerContext.clientDisconnected();         // 在线数 -1
    ServerContext.log("客户端断开 " + addr);
}
```

【✅ 怎么算做好(实操 3 的功能验收点)】运行 `ServerMonitorFrame`,弹出监控窗口、自动启动、状态"运行中"、端口 9000;起一个客户端登录,**在线客户端数从 0 变 1**,日志里出现"客户端接入 ...""... LOGIN → OK";关掉客户端,在线数回 0、日志出现"客户端断开";点"停止"状态变"已停止"、点"启动"又能连。**日志里看不到密码。**

【卡住怎么办】

- **在线数不变 / 日志不动**:`ClientHandler` 接入时要 `ServerContext.clientConnected()`、断开时 `clientDisconnected()`,每处理一条请求要 `ServerContext.log(...)`;监控窗口构造时要 `ServerContext.setListener(this)` 注册。
- **窗口偶尔卡死 / 日志错乱**:多半是没切 EDT,见下面审查第 1 条。

- **点停止后再启动连不上**:`start()` 里要**重新** `new ServerSocket(PORT)`(不能复用已关闭的);`stop()` 要真的 `close()`。

步骤 3.3 · ⭐ 审查:两条铁律,亲自查

生成完**别急着往下走**,逐条审这两条:

审查点	怎么查	没写对怎么补(纠错三步法)
🔴 **跨线程更新 UI**	`onLog` / `onOnlineChange` 里改界面(`logArea.append` / `setText`)有没有包在 `SwingUtilities.invokeLater`	"监控窗口的 `onLog` / `onOnlineChange` 是被客户端线程调用的,直接改界面不安全。请把里面的界面更新包进 `SwingUtilities.invokeLater(...)`,只改这两处。"
🔴 **日志不记密码**	每条请求的日志记的是**命令名**(`LOGIN`)还是**整行**(含用户名密码)	"请求日志不能记完整请求行(含密码)。请只记 `|` 前的命令名和 OK/ERR 结果,别记参数。只改记日志这一处。"

【✅ 怎么算做好(实操 3 的审查验收点)】两条都对:界面更新都在 `invokeLater` 里;日志里看到的是 `LOGIN → OK`,**看不到** `LOGIN|admin|123456`。

> 🔬 **现场验证**:一个客户端登录,盯监控窗口日志——看到 `LOGIN → OK`,不是密码;再开第二个客户端,看**在线数从 1 跳到 2**,关掉一个又回 1。密码没进日志,是你**审查**出来的,不是 AI 天生就会。

> 💬 这两条为什么是铁律?① **Swing 不是线程安全的**(热身 B),别的线程碰界面迟早出诡异 bug,今天不切 EDT 可能"看着没事",但答辩老师一问"这是哪个线程改的界面",你就知道坑在哪。② **日志会落盘、会给人看,密码进日志就是安全事故**。AI 图快很容易两条都漏——**你能看出来、能让它补,这就是你的价值。**

【广度扩展 · 管理端四大功能族(实操 4–7)】

> ⚠️ **先读这一段。** 前面实操 1–3 做的是管理端**主干**(图书 CRUD + 组合搜索 + 监控窗口)。但管理员真正天天用的,还有**统计报表、操作日志、用户审核、分类管理**这四块——它们才是"管理"的重头。
>
> 📌 **和前面不一样的地方:这四块没有答案版代码给你比对**(demo 只做了主干)。所以下面**每一步只给你「该长什么样(预期行为)」+ 审查清单 + 用真实数据验证**,不给"标准答案代码"。**这正是真实开发的常态——大多数功能没有参考答案,你要靠 prompt 说清、靠审查把关、靠数据验证。** 这四块就是练这个的最好机会。
>
> 💡 **每块的审查重头(你判断力的价值点)**:
>
> - **实操 4 统计报表**:审 `GROUP BY` 聚合 SQL 的**统计口径**(报表最会给你一个"看着对其实错"的数字)。
> - **实操 5 操作日志**:写日志**不记密码、异步不阻塞主流程、不进主事务**。
> - **实操 6 用户审核**:"能改状态 ≠ 状态真起作用"——**禁用了还能登录**就是安全漏洞。
> - **实操 7 分类 CRUD**:**删除有关联的数据先问"删了它谁成孤儿"**。

实操 4 · ⭐ 借阅统计报表(练:审 GROUP BY 聚合 SQL)

这一步在做什么

给管理端加三张报表:**分类借阅榜、热门图书榜、库存预警**,放进一个新选项卡 `StatisticsPanel`(三个 `JTable`)。技术核心是 **`GROUP BY` + 聚合 SQL**——这类 SQL 最容易被 AI 写出"能跑、有数字、但口径错"的坑,今天重点练**怎么审对它**。

先分清三种统计口径:

命令(PRD 口径)	统计问题	SQL 核心
`STATS_BORROW_BY_CATEGORY|start_date|end_date`	每个分类借了多少次	按分类 `GROUP BY` + `COUNT`
`STATS_HOT_BOOKS|limit`	借最多的前 N 本	按书 `GROUP BY` + `COUNT` + `ORDER BY DESC LIMIT`
`STATS_STOCK_ALERT|threshold`	可借库存低于阈值的书	**不用** `GROUP BY`,`WHERE available_stock < ?`(是"筛"不是"聚合")

> 🤔 想一想:这三个里**哪个不需要** `GROUP BY`?——库存预警是"筛选"不是"分组聚合"。分清这两个,就抓住了报表 SQL 的第一层。

步骤 4.1 · 复制这段 prompt 给 Trae SOLO

【操作】原样复制输入:

```
基于现有 C/S 图书馆代码,新增管理端「借阅统计报表」,只加统计相关、别动图书/借还/监控窗口。

一、服务端新增三条协议命令(走 dao 层 PreparedStatement,响应 OK|json):
1. STATS_BORROW_BY_CATEGORY|start_date|end_date —— 各分类在时间区间内的借阅次数。
   SQL 思路:FROM t_borrow br JOIN t_book b ON br.book_id=b.id
             JOIN t_category c ON b.category_id=c.id
             WHERE br.borrow_time BETWEEN ? AND ?
             GROUP BY c.id, c.name  ORDER BY 借阅次数 DESC
   返回:[{category_name, borrow_count}]
2. STATS_HOT_BOOKS|limit —— 借阅次数最多的前 limit 本。
   SQL 思路:FROM t_borrow br JOIN t_book b ON br.book_id=b.id
             GROUP BY b.id, b.title  ORDER BY 借阅次数 DESC  LIMIT ?
   返回:[{title, author, borrow_count}]
3. STATS_STOCK_ALERT|threshold —— 可借库存低于阈值的书(不需要 GROUP BY)。
   SQL 思路:FROM t_book WHERE available_stock < ? ORDER BY available_stock ASC
   返回:[{title, available_stock, total_stock}]

二、客户端 StatisticsPanel(加进 AdminMainFrame 新选项卡「统计报表」):
- 顶部:开始/结束日期 + 库存阈值输入 + "刷新报表"按钮;
- 三个 JTable 分区展示三张报表,库存预警那张给低库存行标红/进度条;
- 查询走 Async.run 异步。

约束:PreparedStatement 占位符,不许把日期/阈值拼进 SQL 字符串;空结果显示"暂无数据"不报错。
```

【✅ 怎么算做好】生成了服务端三条 `case` + 统计 dao 方法,客户端 `StatisticsPanel`(三张表 + 输入 + 刷新),`AdminMainFrame` 多了「统计报表」选项卡。

步骤 4.2 · 生成后"该长什么样"(没有答案版,靠行为对照)

- **分类借阅榜**:列「分类 | 借阅次数」,**次数从高到低**;换日期区间点刷新,数字会变。
- **热门图书榜**:列「书名 | 作者 | 借阅次数」,**只显示前 N 行**,降序。
- **库存预警**:列「书名 | 可借库存 | 总库存」,**只列可借库存 < 阈值的书**,库存越低越靠前。
- 三张表在同一个选项卡;刷新不卡界面。

> ⚠️ 这里没有"标准代码"给你抄,**只能靠"数字对不对"判断**——所以下一步的重头是审 SQL。

步骤 4.3 · ⭐ 审查:怎么审对一段 GROUP BY 聚合 SQL

【操作】打开 AI 写的统计 SQL,逐条查:

审查点	怎么查	对的样子	常见错法
🔴 `GROUP BY` 字段全	`SELECT` 的非聚合列(如 `c.name`)是否都在 `GROUP BY` 里	`GROUP BY c.id, c.name`	只 `GROUP BY c.name` 漏 `c.id`
🔴 `COUNT` 数对东西	数的是借阅记录数吗	`COUNT(br.id)`	`COUNT(*)` 在 `LEFT JOIN` 下把没借阅的也数成 1
🔴 `JOIN` 没重复计数	有没有一对多 `JOIN` 把一条借阅数成多条	借阅→书→分类层层多对一	误加一对多表,次数虚高
🔴 排序/阈值方向	热门 `DESC`、库存 `ASC`、阈值是 `<`	各自方向对	库存写成 `> threshold`,列出的全是充足的
🟡 时间用占位符	日期是 `BETWEEN ? AND ?` 还是拼字符串	占位符	拼进 SQL(注入风险)

【加深手法 1 · 让 AI 当你的 SQL 老师】选中分类统计 SQL,追问:

```
逐行解释这段统计 SQL:
1. GROUP BY c.id, c.name 为什么两个字段都要写?只写 c.name 会怎样?
2. COUNT(br.id) 和 COUNT(*) 在这里有区别吗?某分类一次没被借过会显示成 0 吗?
3. WHERE 里的时间过滤改成 HAVING,结果会变吗?为什么?
用中文,举具体数据例子说明。
```

【加深手法 2 · ⭐ 主动制造"错误口径"再审】(本块最值钱的练习)

```
把 STATS_HOT_BOOKS 故意改成一个"能跑但统计口径错"的版本给我看
(比如漏 LIMIT、或 GROUP BY 写错字段、或 COUNT 换成 SUM),先别说错在哪,让我自己判断。
```

判断后用结构化追问纠正:

```
我做了:把热门榜 GROUP BY 从 b.id 改成 b.author。
我期望:统计每本书的借阅次数。
实际:同作者多本书被合并成一行、次数错误累加。
请解释原因,并改回按 b.id, b.title 分组。只改这一处 SQL。
```

【✅ 怎么算做好(审查验收点)】审查五条都过;**用真实数据撞过一次**——手动数种子库里某分类的真实借阅次数,和报表对上;至少抓出并纠正过一个口径错误。

> 🔬 **为什么必须用真实数据验证**:报表**永远给你一个数字**,看着都对。只有拿手数出来的真实次数去撞,才知道口径错没错。**统计功能不能只看"跑出来了"**。

步骤 4.4 · (迭代)加占比

```
迭代(别推倒):分类借阅榜加一列"占比%"=该分类次数/总次数*100(保留一位小数)。只改统计相关。
```

【审查追加】占比的分母(总次数)是不是**同一时间区间**内的?若用了"全部借阅"当分母,占比会失真——自己抓这个坑。

实操 5 · 操作日志审计(练:日志不记密码 / 异步不阻塞)

这一步在做什么

给系统加**持久化的操作审计**:关键操作(登录/借还/增删改/审核)成功后,往 `t_operation_log` 表写一条留痕,管理员能**按时间/用户回溯查询**。

⭐ **先分清:这和实操 3 监控窗口的"实时日志"完全是两回事**:

维度	监控窗口日志(实操 3)	操作日志审计(实操 5)
存哪	**内存**(关了就没)	**数据库**(永久留痕)
能不能查	只能看当前滚动的	**能按时间/用户查询、翻页**
给谁看	运维实时盯"在不在跑"	事后**追溯"谁在什么时候做了什么"**

> 🙋 想一想:只有监控窗口日志,能查出"上周三谁删了《算法导论》"吗?——查不了(内存日志早没了)。**这就是要持久化审计的原因。**

步骤 5.1 · 复制这段 prompt 给 Trae SOLO

【操作】原样复制(注意:`t_operation_log` 表 A2 建库时**没建**,prompt 里要求补建):

```
基于现有 C/S 图书馆代码,新增管理端「操作日志审计」,只加日志相关。

一、补建表(A2 没建这张):
CREATE TABLE t_operation_log(
  id INT PRIMARY KEY AUTO_INCREMENT, user_id INT, user_name VARCHAR(50),
  operation_type VARCHAR(50) NOT NULL, operation_desc VARCHAR(500),
```

```
  target_table VARCHAR(50), target_id INT, ip_address VARCHAR(50),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP);

二、写日志:关键操作成功后记一条,覆盖 登录/借书/还书/续借/图书增删改/用户审核禁用。要求:
- 只记操作类型 + 人话描述(如"删除图书《算法导论》"),绝不把密码/完整请求行写进
operation_desc;
- 用独立线程/线程池异步落库(fire-and-forget),不阻塞主业务;写日志失败只打印告警、不影响主操
作;
- 写日志绝不能放进借书/还书事务里(避免日志失败连累主业务回滚)。

三、查询:LOG_LIST|start_date|end_date|page|size
   SQL:WHERE created_at BETWEEN ? AND ? ORDER BY created_at DESC LIMIT ? OFFSET
?,
   另一条 SELECT COUNT(*) 查总数,响应带回 {列表, 总数}。

四、客户端 OperationLogPanel(AdminMainFrame 加选项卡「操作日志」):
   日期区间 + 查询 + 上/下一页;JTable 列:时间|操作人|操作类型|描述|IP;查询走 Async。

约束:PreparedStatement;operation_desc 里不许出现任何密码。
```

【✅ 怎么算做好】生成了 `t_operation_log` 建表、异步写日志工具、`ClientHandler` 各操作后调写日志、`LOG_LIST` 分支、客户端 `OperationLogPanel`。

步骤 5.2 · "该长什么样"

- 做任意关键操作(登录/借书/删书),`t_operation_log` 就多一条,类型是操作名、描述是人话、**没有密码**。
- 操作日志选项卡按日期区间查得到,**按时间倒序**,能翻页。
- 借书还是那么快(写日志在后台线程)。

步骤 5.3 · ⭐ 审查:审计日志三条底线

审查点	怎么查	没写对怎么补(纠错三步法)
🔴 **不记密码**	翻登录/注册的 `operation_desc` 有没有明文密码	"operation_desc 不能记密码或完整请求行,只记操作类型和人话描述。只改写日志这处。"
🔴 **异步不阻塞**	写日志是不是独立线程;写失败会不会连累主操作	"把写日志改成线程池异步,try-catch 吞掉写日志异常只告警,不影响主操作返回。"
🔴 **日志不进主事务**	借书/还书事务里有没有夹 `INSERT t_operation_log`	"把审计日志 INSERT 移出借书事务,放到事务提交后异步写。"

【加深手法 · ⭐ 制造"日志连累借书"的 bug 再审】

```
故意把 INSERT 审计日志加进借书那个数据库事务里(和减库存、写借阅记录同一事务),先别改回来。
我想看:如果这条日志 INSERT 失败(比如字段超长),借书会不会跟着回滚、用户明明该借成功却报错?
```

观察到"日志失败牵连借书回滚"后:

```
我做了:把审计日志 INSERT 放进了借书事务。
我期望:借书成功就是成功,日志失败也不该影响借书。
实际:日志 INSERT 失败时借书整个回滚了。
请把审计日志移出借书事务,改成事务提交后异步写。只改这一处。
```

【✅ 怎么算做好】三条底线都过;演示过"日志写失败,借书照样成功"。

> 💬 记住:**审计是配角,绝不能喧宾夺主**。日志写失败顶多少一条留痕,但要是把用户借书搞回滚了,就是拿次要功能砸了主功能。这种坑靠点按钮测不出来,必须去审代码。

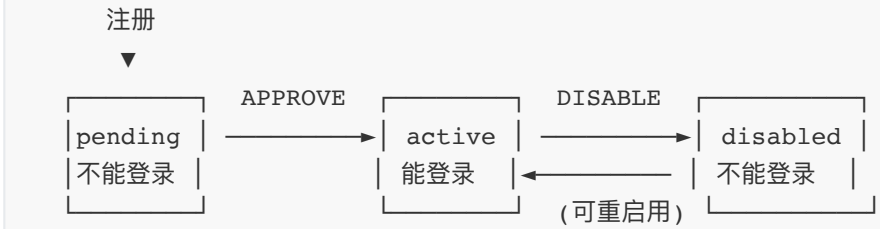
步骤 5.4 · (迭代)按操作人/类型筛

```
迭代(别动已有):LOG_LIST 加可选 user_name 和 operation_type 过滤
(WHERE 动态拼,参照图书组合搜索 WHERE 1=1),客户端加"操作人"输入框和"操作类型"下拉。只改日志相关。
```

实操 6 · ⭐ 用户审核治理(练:能改状态 ≠ 状态起作用)

这一步在做什么

给账号加**三态状态机**:注册默认 `pending`(待审)→ 管理员审核 `USER_APPROVE` 变 `active`(可用)→ 违规 `USER_DISABLE` 变 `disabled`(禁用)。这块直接接上 A2 的登录和注册。



步骤 6.1 · 复制这段 prompt 给 Trae SOLO

【操作】原样复制:

```
基于现有 C/S 图书馆代码,新增管理端「用户审核治理」,接上现有登录/注册链路。

一、状态字段:t_user.status ENUM('pending','active','disabled') 默认 'pending'
   (若没有 status 列先 ALTER TABLE 补上,默认 pending)。

二、服务端命令:
1. USER_APPROVE|user_id —— UPDATE t_user SET status='active' WHERE id=? AND
status='pending';
   返回影响行数,0 行提示"该用户不是待审状态"。
2. USER_DISABLE|user_id —— UPDATE t_user SET status='disabled' WHERE id=?;不允许禁用管理员。
```

```
3. USER_LIST|status|page|size —— 按状态筛(status 空=全部)。

三、⭐ 登录门禁(改 A2 的 LOGIN):只有 status='active' 才允许登录;
   pending 返回 ERR|账号待审核,disabled 返回 ERR|账号已被禁用。

四、⭐ 注册默认待审(改 A2 的 REGISTER):新用户 status 默认 'pending',
   注册成功提示"注册成功,请等待管理员审核"。

五、客户端:用户管理面板加"状态"列 + 状态筛选下拉 + 选中行后的"审核通过""禁用"按钮
   (操作走 Async,成功后刷新)。

约束:PreparedStatement;审核/禁用 UPDATE 必须带 WHERE id=?,绝不能漏 WHERE 批量误改。
```

【✅ 怎么算做好】生成了三条命令、**改了 LOGIN 加 status 校验、改了 REGISTER 默认 pending**、客户端加状态列/筛选/两个按钮。

步骤 6.2 · "该长什么样"

- **注册**:新号 `status=pending`;立刻登录——**被拒**("账号待审核")。
- **审核**:管理员选中它点"审核通过"→ 变 `active`;再登录——**成功**。
- **禁用**:选一个 active 用户点"禁用"→ 变 `disabled`;它再登录——**被拒**("账号已被禁用")。
- 能按状态筛用户列表。

步骤 6.3 · ⭐ 审查:三条安全线

审查点	怎么查	没写对怎么补
🔴 **注册默认 pending**	注册新号查 `status`	"REGISTER 插入时 status 必须默认 'pending'。"
🔴 **未审核不能登录**	用 pending/disabled 登录	"LOGIN 要加 status='active' 校验,pending/disabled 分别返回不同 ERR。"
🔴 **禁用即时生效**	禁用 active 用户后它再登录	"禁用后新的登录要即时读到最新 status,不能有缓存放行。"
🟡 **UPDATE 带 WHERE id**	审核/禁用 UPDATE 有没有 `WHERE id=?`	"审核/禁用 UPDATE 必须带 WHERE id=?,漏了会批量误改所有人。"

【加深手法 · ⭐ 制造"禁用了还能登录"的安全漏洞再审】(本块最典型的坑)

```
检查:如果 USER_DISABLE 把用户改成 disabled,但 LOGIN 的 SQL 只校验 username 和 password、
没校验 status,这个被禁用的用户还能不能登录?先别改,告诉我这是不是安全漏洞。
```

确认是漏洞后:

```
我做了:禁用了用户 test,但发现它还能登录。
我期望:被禁用/待审的用户一律登不上。
实际:LOGIN 没校验 status,禁用没生效。
请在 LOGIN 里加 status='active' 校验,非 active 分别返回"待审核""已禁用"。只改登录校验。
```

【✅ 怎么算做好】三条安全线都过;演示过"禁用漏洞"并修复。

> 💬 这是安全审查最经典的一课:**"能改状态"和"状态真起作用"是两回事**。AI 很容易给你做个"点了禁用、列表也变灰"的功能,让你以为搞定了,结果登录那头根本没拦。**开关和门禁必须都在,少一个就是漏洞**——靠点按钮测不出来,必须审登录 SQL。

步骤 6.4 · (迭代)禁用带原因 + 批量审核

```
迭代(别推倒):USER_DISABLE 加可选 reason,禁用原因记进操作日志(接上实操 5);
用户列表加"批量审核通过",但每条仍走带 WHERE id=? 的单条 UPDATE(循环),
不许写成一条无 WHERE 的批量 UPDATE。只改用户治理相关。
```

实操 7 · 分类 CRUD(练:删除关联数据先问"谁成孤儿")

这一步在做什么

给管理端加**分类管理** `CategoryManagePanel`——它就是**图书 CRUD 换个业务再来一遍**(还记得"管理端 = 同一条链路换业务"吗?连界面套路都一样:JTable + 工具栏 + JDialog)。但藏着一个新问题:**删一个"下面还挂着图书"的分类,必须拦截**,不能让图书的 `category_id` 悬空。

步骤 7.1 · 复制这段 prompt 给 Trae SOLO

【操作】原样复制:

```
基于现有代码,给管理端加「分类管理」CategoryManagePanel(照图书管理 BookManagePanel 的套路,
换成分类业务),加进 AdminMainFrame 新选项卡「分类管理」。只改分类相关。

服务端命令:
- CATEGORY_LIST —— 列出所有分类(id/name/description/该分类下图书数量)。
- CATEGORY_ADD|name|description —— name 唯一,重名返回 ERR|分类名已存在。
- CATEGORY_UPDATE|id|name|description。
- CATEGORY_DELETE|id —— ⭐ 删除前先查该分类下有没有图书
  (SELECT COUNT(*) FROM t_book WHERE category_id=?),有图书则拒绝删除,
  返回 ERR|该分类下还有N本图书,不能删除;无图书才真正删。

客户端:JTable(分类名/描述/图书数量)+ 工具栏(新增/编辑/删除/刷新),
新增/编辑用 JDialog 表单,删除有确认框,操作走 Async 异步。

约束:PreparedStatement;删除必须先做关联检查再删。
```

【✅ 怎么算做好】生成了分类增删改查(含关联计数)、`ClientHandler` 四条分支、客户端 `CategoryManagePanel` + 表单对话框 + 新选项卡。

步骤 7.2 · "该长什么样"

- **界面**:和图书管理**一模一样的套路**(表格 + 工具栏 + 弹表单 + 删除确认)。
- **新增重名**:加已存在的分类名,被拒("分类名已存在")。
- **删除拦截**:删"图书数量>0"的分类,被拒("该分类下还有 N 本图书,不能删除");删空分类,成功。

步骤 7.3 · ⭐ 审查:关联完整性 + CRUD 基本盘

审查点	怎么查	没写对怎么补
🔴 **删除关联拦截**	删一个下面有书的分类	"CATEGORY_DELETE 前要 SELECT COUNT(*) FROM t_book WHERE category_id=?,>0 就拒绝。只改删除这处。"
🟡 **name 唯一**	加重名分类	"CATEGORY_ADD 处理 name 唯一冲突,重名返回友好 ERR,别直接抛 SQL 异常。"
🟡 **表格只读/删除确认/异步**	同图书管理三条(实操 1 学过)	照实操 1 补

【加深手法 · 制造"删除孤儿"的 bug 再审】

```
故意把 CATEGORY_DELETE 改成"不做关联检查、直接 DELETE FROM t_category WHERE id=?",先别改回来。
我想看:删掉一个下面还有 20 本书的分类后,那 20 本书的 category_id 会变成什么?
它们在图书列表的"分类"列会显示成什么?
```

观察到"图书分类悬空"后:

```
我做了:删了有图书的分类,没做关联检查。
我期望:有图书的分类不许删。
实际:图书 category_id 悬空,图书列表分类列变空/报错。
请在 CATEGORY_DELETE 前加关联检查:该分类下有图书就拒绝删除。只改删除逻辑。
```

【让 AI 讲两种方案】

```
拦截"删除有关联的分类"有两种做法:(A) 数据库外键约束;(B) 应用层软删除(加 is_deleted 标记)。
请对比优缺点,以及我们这个 JDBC+MySQL 项目更适合哪种、为什么。用中文。
```

【✅ 怎么算做好】删有书的分类被拦、重名被拒、CRUD 三条审查过;演示过"删除孤儿"并修复。

> 💬 **关联数据删除的通用心法**:删任何"被别人依赖"的东西前,先问"删了它,谁会成孤儿"。分类被图书依赖、用户被借阅记录依赖……到处是这个模式。今天在分类上练熟,课后任务里"删还有借阅记录的读者"你就会先想这一层。

步骤 7.4 · (迭代)合并分类

```
迭代(别推倒):加"合并"——CATEGORY_MERGE|from_id|to_id:把 from 分类下所有图书的
category_id 改成 to,再删掉空掉的 from 分类(两步放一个事务,要么全成要么全不动)。
客户端选中两个分类点"合并到"。只改分类相关。
```

【审查追加】合并的"改 category_id + 删旧分类"两步**是不是在一个事务里**?不是的话中途失败会留下不一致——联系 A3 借书事务的道理自己审。

四大功能族 · 你练到的真功夫(小结)

功能族	补上的能力	审查真功夫
实操 4 统计报表	分类榜/热门书/库存预警	**审 `GROUP BY` 聚合 SQL 口径 + 真实数据验证**
实操 5 操作日志	持久化留痕、可回溯	**不记密码 + 异步不阻塞 + 日志不进主事务**
实操 6 用户审核	三态状态机	**"能改状态 ≠ 状态起作用"的禁用漏洞**
实操 7 分类 CRUD	分类增删改 + 关联拦截	**删关联数据先问"谁成孤儿"**

> 💬 这四块练的是**同一件事的四种样子——AI 能很快把功能"做出来",但"做对"要靠你审**。而且这四块**没有一块有标准答案代码给你抄**,全靠你把需求说清、把结果审明白。这就是"范式 2"里你最值钱的能力:**不是写代码,是判断 AI 写的对不对**。

🔬 拓展理解:两个"同一道理的两面"

今天有两处知识,其实和单元③学过的东西是"同一道理的两面",连起来记更牢:

- **管理端 = 读者端换业务**:读者端"发字符串 → 查库 → 返字符串",管理端也是这一套,命令从"查/借"换成"增删改"。**主干链路只有一条,功能都是复制它。**
- **EDT 铁律的两面**:单元③客户端"耗时的网络请求放 SwingWorker,别在界面线程上干,免得卡界面";今天监控窗口"别的线程别直接改界面,要改先 `invokeLater` 回 EDT"。合起来就是一条原则:**EDT 只做刷界面这一件事,耗时活和别的线程都别掺和进来。**

今天做完了什么(自检清单)

- [ ] 跑通三个热身 demo:进阶控件(表格/弹窗/选项卡+mini-CRUD)、EDT、发布订阅(热身 A/B/C)
- [ ] 管理员端能用:登录 admin 进管理端主窗,图书 CRUD(含删除确认)、读者列表、借阅记录 / 超期统计都在(实操 1)
- [ ] 管理端三条审查过了:表格只读、删除确认、异步请求(实操 1)
- [ ] 图书查询能按关键词 + 分类组合筛;理解了分页怎么加(实操 2)
- [ ] 服务端监控窗口能用:看端口 / 在线数 / 实时日志、能启停;起客户端时在线数 ±1、日志有记录(实操 3)
- [ ] 监控窗口两条铁律审查过了:界面更新切 EDT、日志不记密码(实操 3)

- [ ] 借阅统计报表出数:分类榜 / 热门书 / 库存预警;审过 `GROUP BY` 聚合 SQL 口径、用真实数据撞过、抓出过一个口径错(实操 4)
- [ ] 操作日志审计能用:关键操作留痕、按日期翻页倒序查;审过"不记密码 / 异步不阻塞 / 日志不进主事务"三条底线(实操 5)
- [ ] 用户审核治理能用:注册默认待审、pending/disabled 登不上、审核后能登、禁用即时生效;演示过"禁用漏洞"并修复(实操 6)
- [ ] 分类 CRUD 能用:增删改 + 重名拦截 + 删有书分类被拦;演示过"删除孤儿"并修复(实操 7)

都打勾,今天的目标就达成了——**系统从"读者能用"长成了"管理员能管(能管书、看报表、查审计、审用户、理分类)、后台看得见",而且关键处都由你审查把关过。**

课后任务

给管理端**读者管理**补上"新增 / 编辑 / 删除读者"(答案版这一块留的是只读列表,正好留给你按已有套路扩展)。

怎么做:

1. 先想清楚要加哪些协议命令(参照图书的 `BOOK_ADD` / `BOOK_UPDATE` / `BOOK_DELETE`,给读者设计 `READER_ADD` / `READER_UPDATE` / `READER_DELETE`)。
2. 给 SOLO 一段 prompt(照抓手2 的套路,把"图书"换成"读者"):

   ```
   基于现有代码,给管理员端的读者管理加增删改:协议 READER_ADD|json、READER_UPDATE|json、READER_DELETE|id(服务端走 UserDao),新增/编辑用 JDialog 表单(用户名/姓名/角色/可借上限),删除有确认框。只改读者管理相关,别动图书和借还书。
   ```

3. 生成后**自己审查**——对照实操 1 的管理端三条(表格只读、删除确认、异步请求),缺的用纠错三步法让 AI 补。
4. (挑战)想一想:删除一个"还有未还借阅记录"的读者,应该允许吗?如果不允许,服务端该怎么挡?把你的想法写进 prompt 让 AI 实现。

交付:读者管理能增删改、你审查过,下次课带来。

> 记住今天最重要的两句话:**管理端 = 同一条 C/S 链路换业务;别的线程改界面,先 `invokeLater` 回 EDT,日志不记密码。**